

**LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2**

MODUL 14

Sorting



Disusun oleh:

Ahmad Malik Arrayan

109082500116

S1IF-13-02

Asisten Praktikum

Muhammad Agha Zulfadhi

Renisa Assyifa Putri

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

Tugas

Selection Sort

1. Soal 1

Source Code

```
package main

import (
    "fmt"
)

func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i

        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }

        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func main() {
    var n int

    _, err := fmt.Scan(&n)
    if err != nil {
        return
    }
}
```

```

        for i := 0; i < n; i++ {
            var m int

            fmt.Scan(&m)

            rumah := make([]int, m)

            for j := 0; j < m; j++ {
                fmt.Scan(&rumah[j])
            }

            selectionSort(rumah)

            for j := 0; j < m; j++ {
                if j > 0 {
                    fmt.Print(" ")
                }
                fmt.Print(rumah[j])
            }

            fmt.Println()
        }
    }
}

```

Screenshoot program

The screenshot shows a VS Code editor with a Go file named `Soal1.go`. The code implements a selection sort algorithm. The terminal output shows the execution of the program, displaying the input array and the sorted array.

```

22 func main() {
23     return
24 }
25
26 for i := 0; i < n; i++ {
27     var m int
28     fmt.Scan(&m)
29     rumah := make([]int, m)
30     for j := 0; j < m; j++ {
31         fmt.Scan(&rumah[j])
32     }
33     selectionSort(rumah)
34     for j := 0; j < m; j++ {
35         if j > 0 {
36             fmt.Print(" ")
37         }
38         fmt.Print(rumah[j])
39     }
40     fmt.Println()
41 }
42 }

```

Terminal Output:

```

PS C:\Tugas\laprak 12> go run "C:\Tugas\laprak 12\Soal 1\Soal1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS C:\Tugas\laprak 12>

```

Deskripsi program

Program ini adalah aplikasi berbasis konsol yang ditulis menggunakan bahasa Go (Golang) untuk mengurutkan sekumpulan data angka secara *ascending* (dari nilai terkecil ke terbesar).

Secara alur kerja, program menerima masukan berupa beberapa *test case*. Pada setiap *test case*, program akan meminta input jumlah data yang merepresentasikan "ukuran rumah", dilanjutkan dengan input deretan angka ukurannya. Proses pengurutan utama dilakukan menggunakan algoritma **Selection Sort**. Algoritma ini bekerja dengan cara memindai data untuk mencari nilai terkecil dari kumpulan angka yang belum terurut, lalu menukarnya (*swapping*) secara langsung ke posisi paling depan. Proses ini diulang hingga seluruh rentetan angka berurutan dengan benar. Terakhir, program mencetak deretan ukuran rumah yang sudah terurut rapi, dipisahkan oleh spasi, dengan setiap *test case* berada di baris baru.

2. Soal 2

Source Code

```
package main

import (
    "fmt"
)

func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}

func selectionSortDesc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        maxIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[maxIdx] {
                maxIdx = j
            }
        }
        arr[i], arr[maxIdx] = arr[maxIdx], arr[i]
    }
}
```

```

    }

    func main() {
        var n int

        _, err := fmt.Scan(&n)
        if err != nil {
            return
        }

        for i := 0; i < n; i++ {
            var m int
            fmt.Scan(&m)

            var ganjil []int
            var genap []int

            for j := 0; j < m; j++ {
                var num int
                fmt.Scan(&num)

                if num%2 != 0 {
                    ganjil = append(ganjil, num)
                } else {
                    genap = append(genap, num)
                }
            }

            selectionSortAsc(ganjil)
            selectionSortDesc(genap)

            first := true

```

```

        for _, val := range ganjil {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(val)
            first = false
        }

        for _, val := range genap {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(val)
            first = false
        }

        fmt.Println()
    }
}

```

Screenshoot program

The screenshot shows a Go IDE with a file named `Soal2.go`. The code implements two selection sort functions: `selectionSortAsc` and `selectionSortDesc`. The `main` function calls `selectionSortAsc` on the array `5 2 1 7 9 13` and `selectionSortDesc` on the array `1 7 9 13 2`. The output of the program is shown in the terminal window at the bottom.

```

1 package main
2
3 import (
4     "fmt"
5 )
6
7 func selectionSortAsc(arr []int) {
8     n := len(arr)
9     for i := 0; i < n-1; i++ {
10         minidx := i
11         for j := i+1; j < n; j++ {
12             if arr[j] < arr[minidx] {
13                 minidx = j
14             }
15         }
16         arr[i], arr[minidx] = arr[minidx], arr[i]
17     }
18 }
19
20 func selectionSortDesc(arr []int) {
21     n := len(arr)
22     for i := 0; i < n-1; i++ {
23         maxidx := i
24         for j := i+1; j < n; j++ {
25             if arr[j] > arr[maxidx] {
26                 maxidx = j
27             }
28         }
29         arr[i], arr[maxidx] = arr[maxidx], arr[i]
30     }
31 }
32
33 func main() {
34     arr1 := []int{5, 2, 1, 7, 9, 13}
35     selectionSortAsc(arr1)
36     fmt.Println(arr1)
37
38     arr2 := []int{1, 7, 9, 13, 2}
39     selectionSortDesc(arr2)
40     fmt.Println(arr2)
41 }

```

```

PS C:\Tugas\laprak 12> go run "C:\Tugas\laprak 12\Soal 2\Soal2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS C:\Tugas\laprak 12>

```

Deskripsi program

Program ini adalah aplikasi berbasis konsol yang ditulis menggunakan bahasa Go (Golang) untuk memisahkan dan mengurutkan sekumpulan deret angka berdasarkan sifat ganjil dan genapnya, dengan aturan pengurutan yang berbeda untuk masing-masing kelompok.

Secara alur kerja, program menerima masukan berupa beberapa *test case*. Pada setiap *test case*, program akan meminta input jumlah angka, kemudian membaca nilai angka tersebut satu per satu. Saat dibaca, program langsung mengevaluasi setiap angka: jika bernilai ganjil, angka tersebut dimasukkan ke dalam *slice* ganjil, dan jika genap, dimasukkan ke dalam *slice* genap.

Setelah pemisahan kelompok selesai, program melakukan proses pengurutan menggunakan algoritma **Selection Sort**. Terdapat dua variasi fungsi pengurutan yang digunakan dalam program ini:

- `selectionSortAsc`: Digunakan untuk mengurutkan kelompok angka ganjil secara menaik (*ascending* / dari terkecil ke terbesar).
- `selectionSortDesc`: Digunakan untuk mengurutkan kelompok angka genap secara menurun (*descending* / dari terbesar ke terkecil).

Terakhir, program mencetak hasilnya ke layar dalam satu baris untuk setiap *test case*. Deretan angka ganjil yang sudah terurut menaik akan dicetak terlebih dahulu, langsung disusul oleh deretan angka genap yang terurut menurun, di mana setiap angka dipisahkan dengan rapi oleh sebuah spasi.

3. Soal 3

Source Code

```
package main

import (
    "fmt"
)

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func main() {
    var data []int
    var num int

    for {
        _, err := fmt.Scan(&num)
        if err != nil {
            break
        }
    }
}
```

```

        if num == -5313 {
            break
        }

        if num == 0 {
            if len(data) > 0 {
                insertionSort(data)

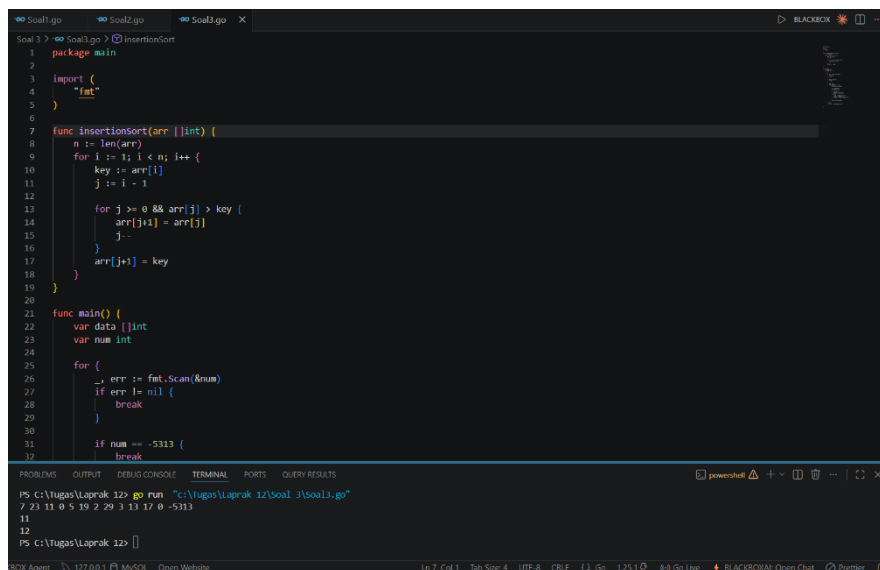
                n := len(data)
                var median int

                if n%2 != 0 {
                    median = data[n/2]
                } else {
                    mid1 := data[(n/2)-1]
                    mid2 := data[n/2]
                    median = (mid1 + mid2) / 2
                }

                fmt.Println(median)
            }
        } else {
            data = append(data, num)
        }
    }
}

```

Screenshoot program



```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func insertionSort(arr []int) {
8     n := len(arr)
9     for i := 1; i < n; i++ {
10         key := arr[i]
11         j := i - 1
12         for j >= 0 && arr[j] > key {
13             arr[j+1] = arr[j]
14             j--
15         }
16         arr[j+1] = key
17     }
18 }
19
20
21 func main() {
22     var data []int
23     var num int
24
25     for {
26         err := fmt.Scan(&num)
27         if err != nil {
28             break
29         }
30         if num == 0 || num == -5313 {
31             break
32         }
33     }
34
35     insertionSort(data)
36     for _, v := range data {
37         fmt.Print(v, " ")
38     }
39     fmt.Println()
40 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS

```
PS C:\Tugas\Laparak 12> go run "C:\Tugas\Laparak 12\Soal 3\Soal3.go"
2 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12
PS C:\Tugas\Laparak 12>
```

Deskripsi program

Program ini adalah aplikasi berbasis konsol yang ditulis menggunakan bahasa Go (Golang) untuk mengumpulkan aliran data (*data stream*) berupa bilangan bulat secara dinamis, lalu menghitung dan menampilkan nilai tengah (*median*) dari kumpulan data tersebut setiap kali dipicu oleh pengguna.

Secara alur kerja, program membaca masukan angka secara terus-menerus menggunakan *infinite loop*. Evaluasi masukan dibagi ke dalam tiga aturan utama:

1. **Kondisi Terminasi (Berhenti):** Jika program menerima input angka **-5313** (berfungsi sebagai *sentinel value*) atau jika tidak ada lagi data yang bisa dibaca (EOF), program akan langsung berhenti.
2. **Kondisi Kalkulasi (Trigger):** Jika program menerima input angka **0**, program akan mulai memproses seluruh data yang sudah terkumpul sejauh itu. Data akan diurutkan terlebih dahulu secara menaik (*ascending*) menggunakan algoritma **Insertion Sort**. Setelah terurut, program mencari nilai *median*. Jika jumlah elemen ganjil, program mengambil elemen tepat di tengah. Jika genap, program mengambil rata-rata dari dua elemen tengah (menggunakan pembagian bilangan bulat / *integer division*). Hasil *median* ini kemudian dicetak ke layar.
3. **Kondisi Akumulasi:** Jika input yang diberikan adalah angka selain 0 dan -5313, angka tersebut akan terus ditambahkan (*append*) ke dalam *slice* data.

Sebagai catatan penting dari perilaku program ini: kumpulan data tidak dikosongkan setelah angka 0 dipanggil. Ini berarti setiap kali angka 0 dimasukkan, program akan mengurutkan dan menghitung *median* dari *seluruh* akumulasi angka yang telah diinput sejak program pertama kali dijalankan.

Insertion Sort

1. Soal 1

Source Code

```
package main

import (
    "fmt"
)

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
        key := arr[i]
        j := i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func main() {
    var data []int
    var num int

    for {
        _, err := fmt.Scan(&num)
        if err != nil {
            break
        }
        if num < 0 {
            break
        }
    }
}
```

```

    }

    data = append(data, num)
}

if len(data) > 0 {
    insertionSort(data)

    for i, val := range data {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(val)
    }
    fmt.Println()

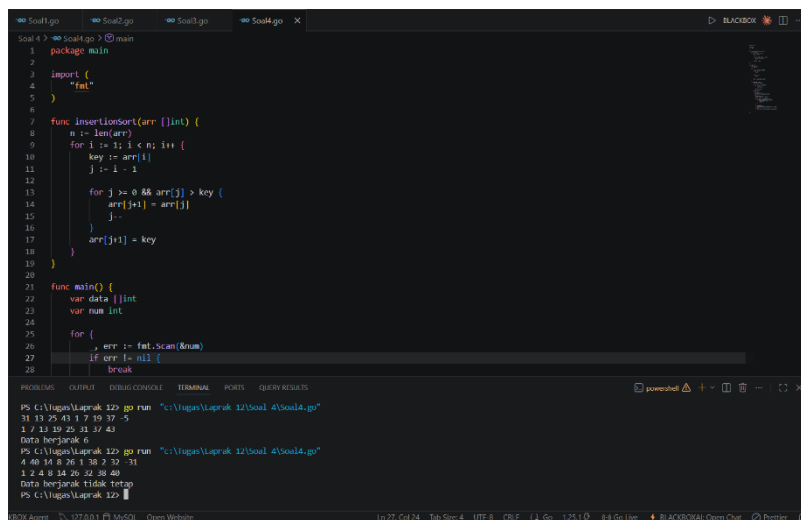
    if len(data) == 1 {
        fmt.Println("Data berjarak 0")
    } else {
        jarakTetap := true
        jarak := data[1] - data[0]

        for i := 2; i < len(data); i++ {
            if data[i]-data[i-1] != jarak {
                jarakTetap = false
                break
            }
        }

        if jarakTetap {
            fmt.Printf("Data berjarak %d\n", jarak)
        } else {
            fmt.Println("Data berjarak tidak tetap")
        }
    }
}
}

```

Screenshoot program



```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 func InsertionSort(arr []int) {
8     n := len(arr)
9     for i := 1; i < n; i++ {
10         key := arr[i]
11         j := i - 1
12
13         for j >= 0 && arr[j] > key {
14             arr[j+1] = arr[j]
15             j--
16         }
17         arr[j+1] = key
18     }
19 }
20
21 func main() {
22     var data []int
23     var run int
24
25     for {
26         i, err := fmt.Scan(&num)
27         if err != nil {
28             break
29         }
30     }
31
32     InsertionSort(data)
33     fmt.Println(data)
34
35     if len(data) == 1 {
36         fmt.Println("Data berjarak 0")
37     } else {
38         distance := data[1] - data[0]
39         for i := 2; i < len(data); i++ {
40             if data[i] - data[i-1] != distance {
41                 fmt.Println("Data berjarak tidak tetap")
42                 return
43             }
44         }
45         fmt.Println("Data berjarak", distance)
46     }
47 }
```

PS C:\Users\Laparak 12> go run "C:\Users\Laparak 12\soal 4\soal4.go"

31 33 25 43 1 7 19 37 -5

Data berjarak 6

PS C:\Users\Laparak 12> go run "C:\Users\Laparak 12\soal 4\soal4.go"

4 40 16 8 26 1 38 2 52 -31

Data berjarak tidak tetap

PS C:\Users\Laparak 12>

Deskripsi program

Program ini adalah aplikasi berbasis konsol yang ditulis menggunakan bahasa Go (Golang) untuk mengumpulkan sekumpulan bilangan bulat, mengurutkannya, dan menganalisis apakah jarak (selisih) antar angka di dalam kumpulan tersebut konstan (membentuk deret aritmatika) atau tidak.

Secara alur kerja, program akan terus menerima input angka secara berulang. Proses pembacaan data ini akan langsung berhenti jika pengguna memasukkan angka negatif (berfungsi sebagai *sentinel value*) atau jika input sudah habis dibaca (EOF). Angka negatif tersebut tidak akan dimasukkan ke dalam kumpulan data.

Setelah proses input selesai dan jika ada data yang tersimpan, program melakukan dua tahap pemrosesan utama:

1. **Pengurutan Data:** Data diurutkan dari nilai terkecil ke terbesar (*ascending*) menggunakan algoritma **Insertion Sort**. Hasil pengurutan ini langsung dicetak ke layar dalam satu baris dengan jarak satu spasi antar angkanya.
2. **Evaluasi Jarak:** Program kemudian mengecek selisih antar elemen yang berdekatan.
 - o Jika kumpulan data hanya berisi satu angka, program akan mencetak "Data berjarak 0".
 - o Jika terdapat lebih dari satu angka, program menjadikan selisih antara elemen pertama dan kedua sebagai acuan. Program lalu membandingkan acuan ini dengan selisih pada seluruh pasangan angka berikutnya.
 - o Jika semua pasangan memiliki selisih yang konsisten sama, program akan mencetak "Data berjarak [nilai jarak]". Namun, jika ditemukan setidaknya satu selisih yang melenceng dari acuan, program akan langsung mencetak "Data berjarak tidak tetap".

2. Soal 2

Source Code

```
package main

import (
    "fmt"
)

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
            &pustaka[i].penulis, &pustaka[i].penerbit,
            &pustaka[i].eksemplar, &pustaka[i].tahun,
            &pustaka[i].rating)
    }
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }
}
```

```

maxIdx := 0
for i := 1; i < n; i++ {
    if pustaka[i].rating > pustaka[maxIdx].rating {
        maxIdx = i
    }
}

fmt.Println(pustaka[maxIdx].judul,
pustaka[maxIdx].penulis, pustaka[maxIdx].penerbit,
pustaka[maxIdx].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        key := pustaka[i]
        j := i - 1

        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }
        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5
    if n < 5 {
        limit = n
    }

    for i := 0; i < limit; i++ {
        if i > 0 {

```



```

        fmt.Print(" ")
    }
    fmt.Print(pustaka[i].judul)
}
fmt.Println()
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    left := 0
    right := n - 1
    foundIdx := -1

    for left <= right {
        mid := (left + right) / 2

        if pustaka[mid].rating == r {
            foundIdx = mid
            break
        } else if r > pustaka[mid].rating {
            right = mid - 1
        } else {
            left = mid + 1
        }
    }

    if foundIdx != -1 {
        b := pustaka[foundIdx]
        fmt.Println(b.judul, b.penulis, b.penerbit,
b.tahun, b.eksemplar, b.rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating
seperti itu")
    }
}

```

```

    }

    func main() {

        var Pustaka DaftarBuku

        var nPustaka int

        DaftarkanBuku(&Pustaka, &nPustaka)

        CetakTerfavorit(Pustaka, nPustaka)

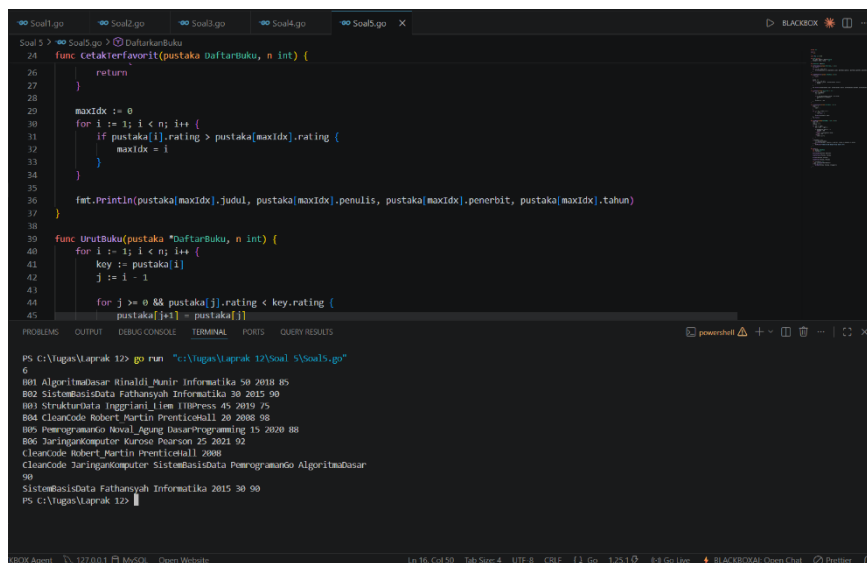
        UrutBuku(&Pustaka, nPustaka)

        Cetak5Terbaru(Pustaka, nPustaka)

        var ratingDicari int
        _, err := fmt.Scan(&ratingDicari)
        if err == nil {
            CariBuku(Pustaka, nPustaka, ratingDicari)
        }
    }
}

```

Screenshoot program



```

Soul5.go  Soul2.go  Soul3.go  Soul4.go  Soul5.go  X
Soul5 > Soul5.go > DaftarkanBuku
24 func cetakterfavorit(pustaka *DaftarBuku, n int) {
25     return
26 }
27
28
29 maxIdx := 0
30 for i := 1; i < n; i++ {
31     if pustaka[i].rating > pustaka[maxIdx].rating {
32         maxIdx = i
33     }
34 }
35
36 fmt.Println(pustaka[maxIdx].judul, pustaka[maxIdx].penulis, pustaka[maxIdx].penerbit, pustaka[maxIdx].tahun)
37 }
38
39 func UrutBuku(pustaka *DaftarBuku, n int) {
40     for i := 1; i < n; i++ {
41         key := pustaka[i]
42         j := i - 1
43         for j >= 0 && pustaka[j].rating < key.rating {
44             pustaka[j+1] = pustaka[j]
45         }
46     }
47 }
48
49
50 PS C:\Vugas\laprak 12> go run "C:\Vugas\laprak 12\Soul 5\Soul5.go"
6
801 AlgoritmaDasar Kiriadi, Amir Informatika 50 2018 85
802 SistemBasisData Fathansyah Informatika 30 2015 90
803 StrukturData Inggrilani, Liew IIBPress 45 2019 75
804 CleanCode Robert Martin PrenticeHall 20 2008 98
805 Pemrograman Noval, Agung DasarProgramming 15 2020 88
806 Jaringankomputer Kurnia Pearson 25 2021 92
CleanCode Robert Martin PrenticeHall 2008
CleanCode Jaringankomputer SistemBasisData Pemrograman AlgoritmaDasar
90
SistemBasisData Fathansyah Informatika 2015 30 90
PS C:\Vugas\laprak 12>

```

Deskripsi program

Program ini adalah aplikasi berbasis konsol yang ditulis menggunakan bahasa Go (Golang) untuk mengelola dan memproses sekumpulan data buku perpustakaan secara komprehensif menggunakan struktur data *array of structs*.

Secara alur kerja, program mengeksekusi serangkaian pemrosesan data secara berurutan melalui pemanggilan berbagai modul (fungsi) di dalam main:

1. **Pendaftaran Buku (DaftarkanBuku):** Program pertama kali menerima input berupa jumlah buku yang akan didaftarkan. Setelah itu, program membaca detail informasi masing-masing buku secara spesifik (ID, judul, penulis, penerbit, jumlah eksemplar, tahun rilis, dan angka rating) lalu menyimpannya ke dalam memori.
2. **Pencarian Terfavorit (CetakTerfavorit):** Program memindai kumpulan data yang ada (*linear search*) untuk menyeleksi satu buku yang memiliki nilai rating paling tinggi, kemudian mencetak informasi buku tersebut (judul, penulis, penerbit, dan tahun).
3. **Pengurutan Data (UrutBuku):** Seluruh data buku kemudian diurutkan berdasarkan nilai ratingnya dari urutan yang paling tinggi hingga terendah (*descending*). Proses ini diimplementasikan menggunakan algoritma **Insertion Sort**.
4. **Mencetak Top 5 (Cetak5Terbaru):** Karena *array* buku sudah berhasil diurutkan dari rating tertinggi pada tahap sebelumnya, program dapat langsung mencetak deretan judul dari maksimal 5 buku teratas. Judul-judul ini dicetak bersebelahan dalam satu baris.
5. **Pencarian Rating Spesifik (CariBuku):** Pada akhir eksekusi, program akan meminta satu masukan angka berupa target rating yang ingin dicari oleh pengguna. Program lalu menelusuri tumpukan data menggunakan algoritma **Binary Search** (yang logikanya sudah disesuaikan untuk mencari pada *array* bersusunan *descending*). Jika ada buku dengan rating yang pas, rincian buku tersebut akan ditampilkan. Jika meleset atau tidak ada yang sesuai, program mencetak peringatan: *"Tidak ada buku dengan rating seperti itu"*.